# Week 8 Report

**Computational Statistics for Data Science · ISS507700**

Student ID: 112006234

April 26, 2026

*I have largely solved the problems without the use of any AI agent or tool.*

## Q1 · Use the interactive_regression() function from CompStatsLib.

---

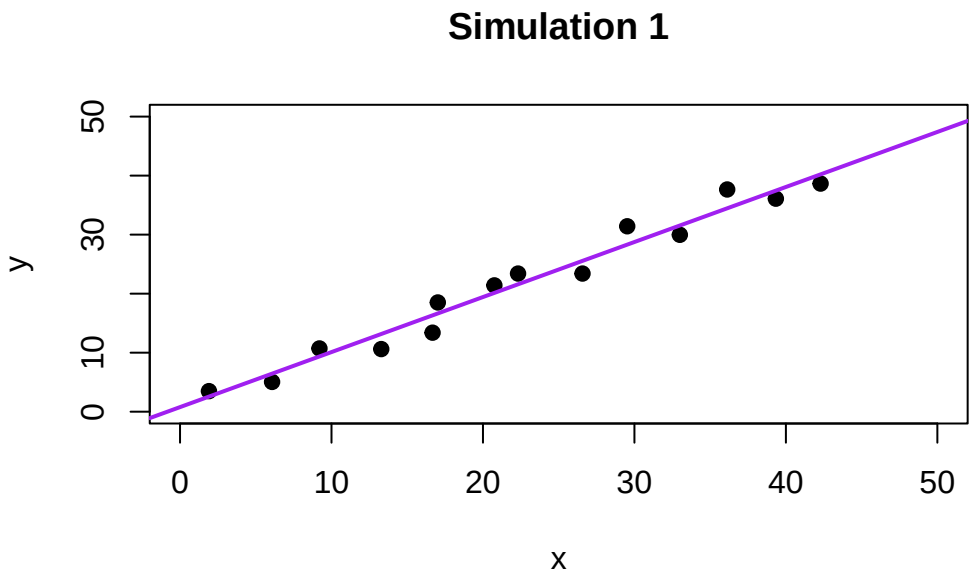
### Simulation of the 4 scenarios

```
library(compstatslib)
#sim_A <- interactive_regression()
#saveRDS(sim_A, "sim_A.rds")
sim_A <- readRDS("sim_A.rds")

# define global limits
xlim <- c(0, 50)
ylim <- c(0, 50)

plot(sim_A$x, sim_A$y,
     pch = 19,
     main = "Simulation 1",
     xlab = "x", ylab = "y",
     xlim = xlim,
     ylim = ylim)

abline(lm(sim_A$y ~ sim_A$x), col = "purple", lwd = 2)
```




```r
# fit model
model <- lm(sim_A$y ~ sim_A$x)

# extract values
intercept <- coef(model)[1]
slope <- coef(model)[2]
r <- cor(sim_A$x, sim_A$y)

y <- sim_A$y
y_hat <- fitted(model)

SSE <- sum((y - y_hat)^2)
SST <- sum((y - mean(y))^2)
SSR <- sum((y_hat - mean(y))^2)

R2 <- SSR / SST

cat(sprintf("Raw intercept: %.2f\n", intercept))
```

```
#> Raw intercept: 0.78
```

```r
cat(sprintf("Raw slope    : %.2f\n", slope))
```

```
#> Raw slope    : 0.93
```

```
cat(sprintf("Correlation  : %.2f\n", r))
```

```
#> Correlation  : 0.98
```

```
cat(sprintf("SSR          : %.2f\n", SSR))
```

```
#> SSR          : 1790.16
```

```
cat(sprintf("SSE          : %.2f\n", SSE))
```

```
#> SSE          : 59.45
```

```
cat(sprintf("SST          : %.2f\n", SST))
```

```
#> SST          : 1849.61
```

```
cat(sprintf("R-squared    : %.2f\n", R2))
```

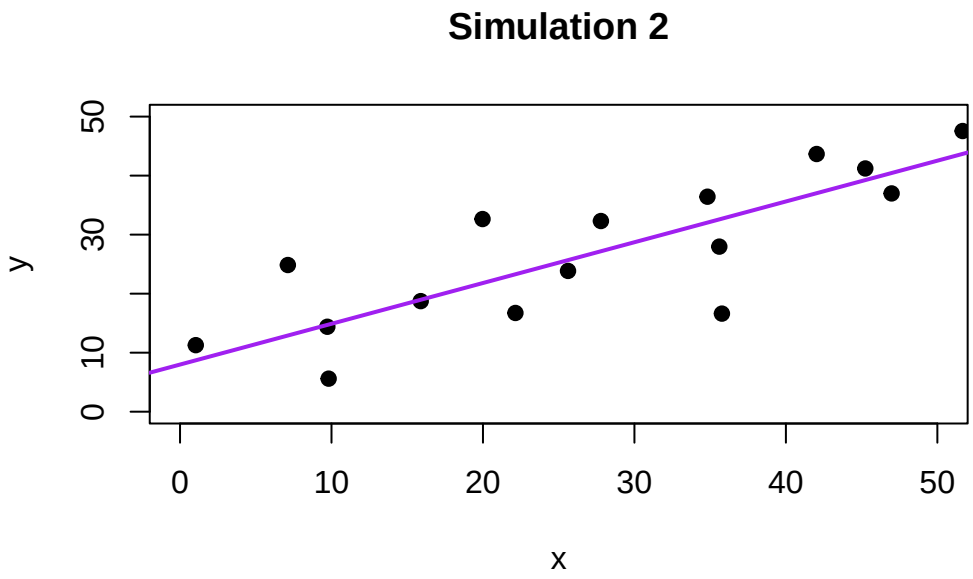
```
#> R-squared    : 0.97
```

```
#sim_B <- interactive_regression()
#saveRDS(sim_B, "sim_B.rds")
sim_B <- readRDS("sim_B.rds")

# define global limits
xlim <- c(0, 50)
ylim <- c(0, 50)

plot(sim_B$x, sim_B$y,
     pch = 19,
     main = "Simulation 2",
     xlab = "x", ylab = "y",
     xlim = xlim,
     ylim = ylim)

abline(lm(sim_B$y ~ sim_B$x), col = "purple", lwd = 2)
```




```r
# fit model
model <- lm(sim_B$y ~ sim_B$x)

# extract values
intercept <- coef(model)[1]
slope <- coef(model)[2]
r <- cor(sim_B$x, sim_B$y)

y <- sim_B$y
y_hat <- fitted(model)

SSE <- sum((y - y_hat)^2)
SST <- sum((y - mean(y))^2)
SSR <- sum((y_hat - mean(y))^2)

R2 <- SSR / SST

cat(sprintf("Raw intercept: %.2f\n", intercept))
```

```
#> Raw intercept: 7.99
```

```r
cat(sprintf("Raw slope    : %.2f\n", slope))
```

```
#> Raw slope    : 0.69
```

```r
cat(sprintf("Correlation  : %.2f\n", r))
```

```
#> Correlation  : 0.85
```

```r
cat(sprintf("SSR          : %.2f\n", SSR))
```

```
#> SSR          : 2179.14
```

```r
cat(sprintf("SSE          : %.2f\n", SSE))
```

```
#> SSE          : 824.23
```

```r
cat(sprintf("SST          : %.2f\n", SST))
```

```
#> SST          : 3003.37
```

```r
cat(sprintf("R-squared    : %.2f\n", R2))
```

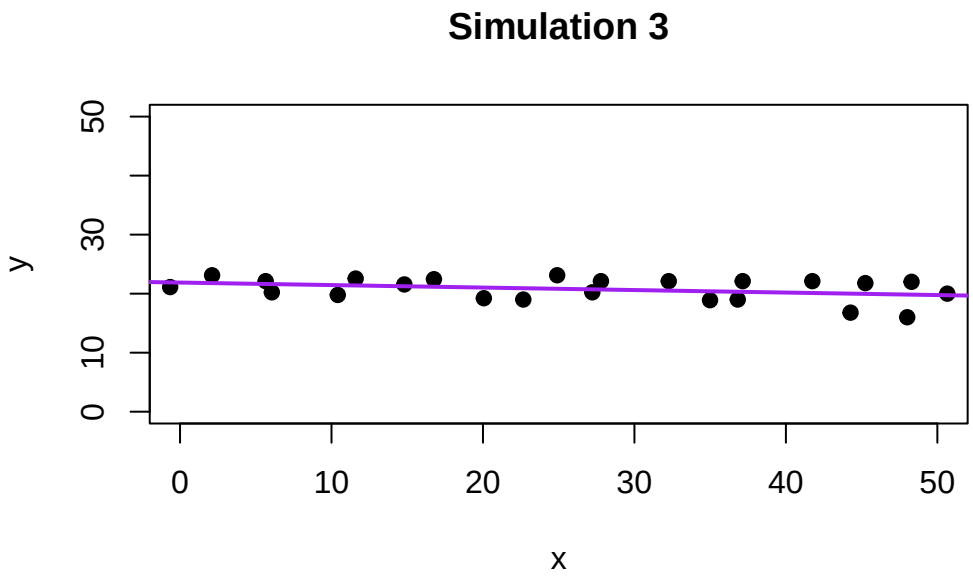
```
#> R-squared    : 0.73
```

```r
#sim_C <- interactive_regression()
#saveRDS(sim_C, "sim_C.rds")
sim_C <- readRDS("sim_C.rds")

# define global limits
xlim <- c(0, 50)
ylim <- c(0, 50)

plot(sim_C$x, sim_C$y,
     pch = 19,
     main = "Simulation 3",
     xlab = "x", ylab = "y",
     xlim = xlim,
     ylim = ylim)

abline(lm(sim_C$y ~ sim_C$x), col = "purple", lwd = 2)
```




```r
# fit model
model <- lm(sim_C$y ~ sim_C$x)

# extract values
intercept <- coef(model)[1]
slope <- coef(model)[2]
r <- cor(sim_C$x, sim_C$y)

y <- sim_C$y
y_hat <- fitted(model)

SSE <- sum((y - y_hat)^2)
SST <- sum((y - mean(y))^2)
SSR <- sum((y_hat - mean(y))^2)

R2 <- SSR / SST

cat(sprintf("Raw intercept: %.2f\n", intercept))
```

```
#> Raw intercept: 21.88
```

```r
cat(sprintf("Raw slope    : %.2f\n", slope))
```

```
#> Raw slope    : -0.04
```

```
cat(sprintf("Correlation  : %.2f\n", r))
```

```
#> Correlation  : -0.39
```

```
cat(sprintf("SSR          : %.2f\n", SSR))
```

```
#> SSR          : 13.35
```

```
cat(sprintf("SSE          : %.2f\n", SSE))
```

```
#> SSE          : 73.82
```

```
cat(sprintf("SST          : %.2f\n", SST))
```

```
#> SST          : 87.18
```

```
cat(sprintf("R-squared    : %.2f\n", R2))
```

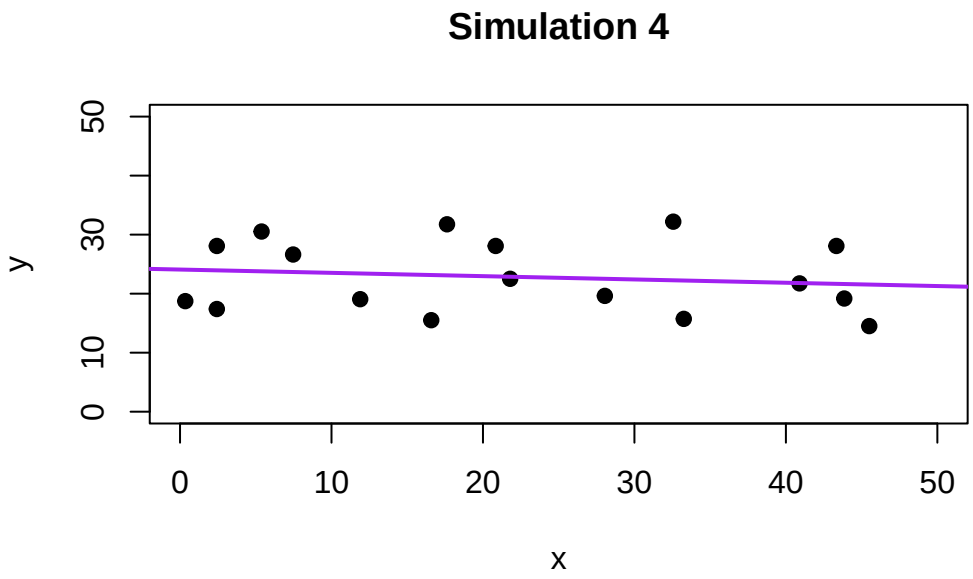
```
#> R-squared    : 0.15
```

```
#sim_D <- interactive_regression()
#saveRDS(sim_D, "sim_D.rds")
sim_D <- readRDS("sim_D.rds")

# define global limits
xlim <- c(0, 50)
ylim <- c(0, 50)

plot(sim_D$x, sim_D$y,
     pch = 19,
     main = "Simulation 4",
     xlab = "x", ylab = "y",
     xlim = xlim,
     ylim = ylim)

abline(lm(sim_D$y ~ sim_D$x), col = "purple", lwd = 2)
```




```r
# fit model
model <- lm(sim_D$y ~ sim_D$x)

# extract values
intercept <- coef(model)[1]
slope <- coef(model)[2]
r <- cor(sim_D$x, sim_D$y)

y <- sim_D$y
y_hat <- fitted(model)

SSE <- sum((y - y_hat)^2)
SST <- sum((y - mean(y))^2)
SSR <- sum((y_hat - mean(y))^2)

R2 <- SSR / SST

cat(sprintf("Raw intercept: %.2f\n", intercept))
```

```
#> Raw intercept: 24.08
```

```r
cat(sprintf("Raw slope    : %.2f\n", slope))
```

```
#> Raw slope    : -0.06
```

```
cat(sprintf("Correlation  : %.2f\n", r))
```

```
#> Correlation  : -0.16
```

```
cat(sprintf("SSR          : %.2f\n", SSR))
```

```
#> SSR          : 15.93
```

```
cat(sprintf("SSE          : %.2f\n", SSE))
```

```
#> SSE          : 591.99
```

```
cat(sprintf("SST          : %.2f\n", SST))
```

```
#> SST          : 607.92
```

```
cat(sprintf("R-squared    : %.2f\n", R2))
```

```
#> R-squared    : 0.03
```

**(a) Comparing scenarios 1 and 2, which do we expect to have a stronger $R^2$?**

Scenario 1

less dispersion → points closer to line → smaller SSE. It can be seen $R^2$ of simulation 1 is 0.97, while simulation 2's is 0.49.

**(b) Comparing scenarios 3 and 4, which do we expect to have a stronger $R^2$?**

Scenario 3

less dispersion → better fit. It can be seen that $R^2$ of simulation 3 is 0.15, while simulation 4's is 0.03.

**(c) Comparing scenarios 1 and 2, which do we expect has bigger/smaller SSE, SSR, and SST??**

Scenario 2 has larger SSE and larger SST due to higher dispersion. SSR is not guaranteed to be larger, as it depends on how much variation is explained by the regression.

Scenario 2 has larger SSE and SST than Scenario 1 due to higher dispersion, while SSR is also larger here in our case, but it may differ since the main difference comes from increased noise (SSE), not the underlying trend. Scenario 1:
SSR: 1790.16
SSE : 59.45
SST: 1849.61

Scenario 2:
SSR: 2179.14
SSE: 824.23
SST: 3003.37

**(d) Comparing scenarios 3 and 4, which do we expect has bigger/smaller SSE, SSR, and SST?**

> Scenario 4 has larger SSE and SST. SSR is similar (not necessarily larger).

Scenario 4 has greater dispersion, so the data points are farther from the regression line, leading to a larger SSE. Since SST measures total variation in the data, it is also larger in Scenario 4. However, SSR depends on how much variation is explained by the regression line. Because both scenarios have a shallow slope, the explained variation is small, so SSR is expected to be similar and is not guaranteed to be larger, even if it may appear slightly larger in this specific simulation.

Scenario 3:
SSR: 13.35
SSE : 73.82
SST: 87.18

Scenario 4:
SSR: 15.93
SSE: 591.99
SST: 607.92

## Q2 · Analyze the programmer_salaries.txt dataset.

---

```
prog <- read.csv("programmer_salaries.txt", sep = "\t")
```

**(a) Use the lm() function to estimate the regression model Salary ~ Experience + Score + Degree. Show the beta coefficients, R2, and the first 5 values of y (*fitted.values*)*and*(residuals)**

```
regr <- lm(Salary ~ Experience + Score + Degree, data = prog)

coef(regr)
```

```
#> (Intercept)  Experience       Score      Degree
#>    7.944849    1.147582    0.196937    2.280424
```

```
summary(regr)$r.squared
```

```
#> [1] 0.8467961
```

```
head(regr$fitted.values, 5)
```

```
#>        1        2        3        4        5
#> 27.89626 37.95204 26.02901 32.11201 36.34251
```

```
head(regr$residuals, 5)
```

```
#>          1          2          3          4          5
#> -3.8962605  5.0479568 -2.3290112  2.1879860 -0.5425072
```

**(b) Use only linear algebra and the geometric view of regression to estimate the regression yourself:**

**(i) Create an X matrix that has a first column of 1s followed by columns of the independent variables. (show the code only)**

```
X <- as.matrix(cbind(1, prog[, c("Experience", "Score", "Degree")]))
```

**(ii) Create a y vector with the Salary values (show the code only).**

```
y <- prog$Salary
```

**(iii) Compute the beta_hat vector of estimated regression coefficients (show the code and values).**

```
beta_hat <- solve(t(X) %*% X) %*% t(X) %*% y
beta_hat
```

```
#>                [,1]
#> 1          7.944849
#> Experience 1.147582
#> Score      0.196937
#> Degree     2.280424
```

**(iv) Using the above, compute a y_hat vector of estimated y values, and a res vector of residuals (show the code and the first 5 values of y_hat and res)**

```
y_hat <- X %*% beta_hat
res <- y - y_hat

head(y_hat, 5)
```

```
#>          [,1]
#> [1,] 27.89626
#> [2,] 37.95204
#> [3,] 26.02901
#> [4,] 32.11201
#> [5,] 36.34251
```

```
head(res, 5)
```

```
#>            [,1]
#> [1,] -3.8962605
#> [2,]  5.0479568
#> [3,] -2.3290112
#> [4,]  2.1879860
#> [5,] -0.5425072
```

**(v) Using only the results from (i) – (iv), compute SSR, SSE and SST (show the code and values)**

```
SSE <- sum(res^2)
SST <- sum((y - mean(y))^2)
SSR <- sum((y_hat - mean(y))^2)

SSE
```

```
#> [1] 91.88949
```

```
SSR
```

```
#> [1] 507.896
```

```
SST
```

```
#> [1] 599.7855
```

**(c) Compute $R^2$ in two ways, and confirm you get the same results (show code and values):**

**(i) Use any combination of SSR, SSE, and SST**

```
R2_1 <- SSR / SST
R2_1
```

```
#> [1] 0.8467961
```

**(ii) Use the squared correlation of vectors y and y**

```
R2_2 <- cor(y, y_hat)^2
R2_2
```

```
#>           [,1]
#> [1,] 0.8467961
```

## Q3 · Look at the data in file auto-data.txt.

---

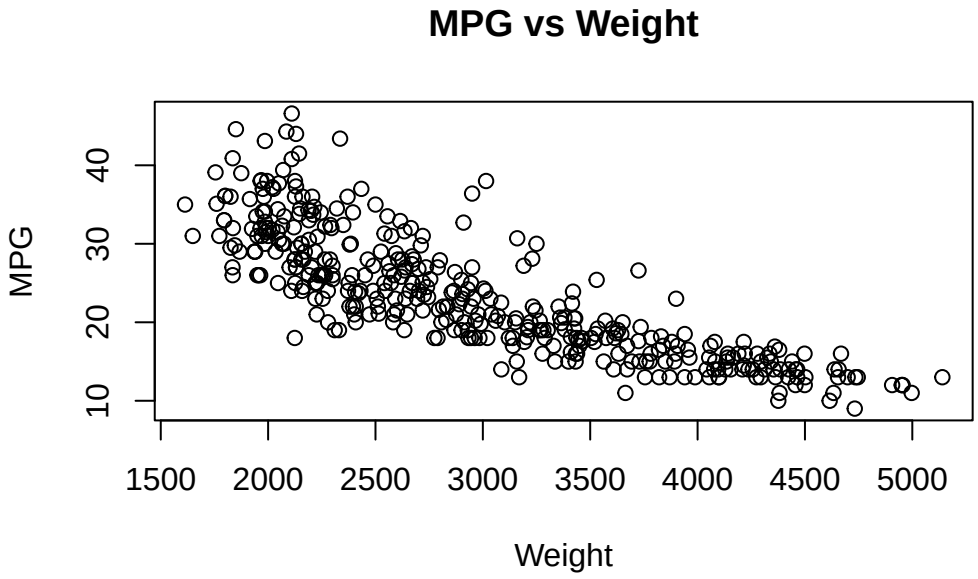
**(a) Try exploring data and problem:**

**(i) Visualize the data as you wish to answer the following questions (report only relevant plots)**

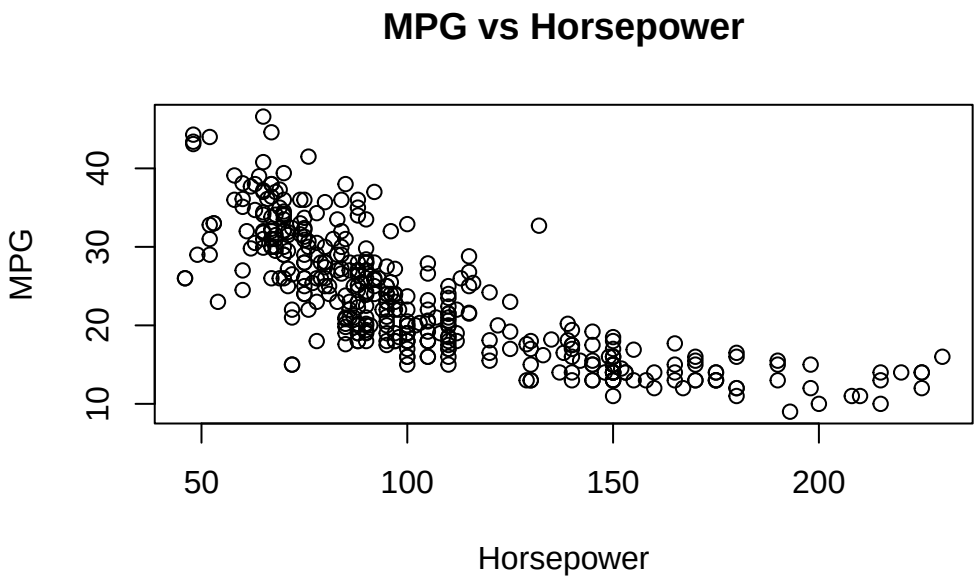
```
auto <- read.table("auto-data.txt", header = FALSE, na.strings = "?")
names(auto) <- c("mpg", "cylinders", "displacement", "horsepower", "weight",
                 "acceleration", "model_year", "origin", "car_name")

auto_num <- auto[, c("mpg", "cylinders", "displacement", "horsepower",
                     "weight", "acceleration", "model_year", "origin")]
```

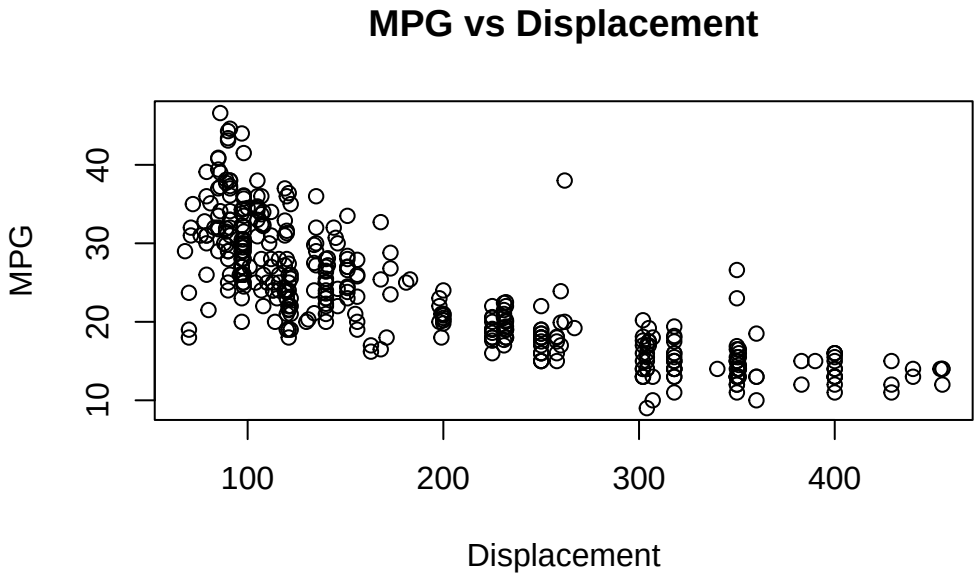
```
# relevant plots
plot(auto$weight, auto$mpg,
     xlab = "Weight", ylab = "MPG",
     main = "MPG vs Weight")
```



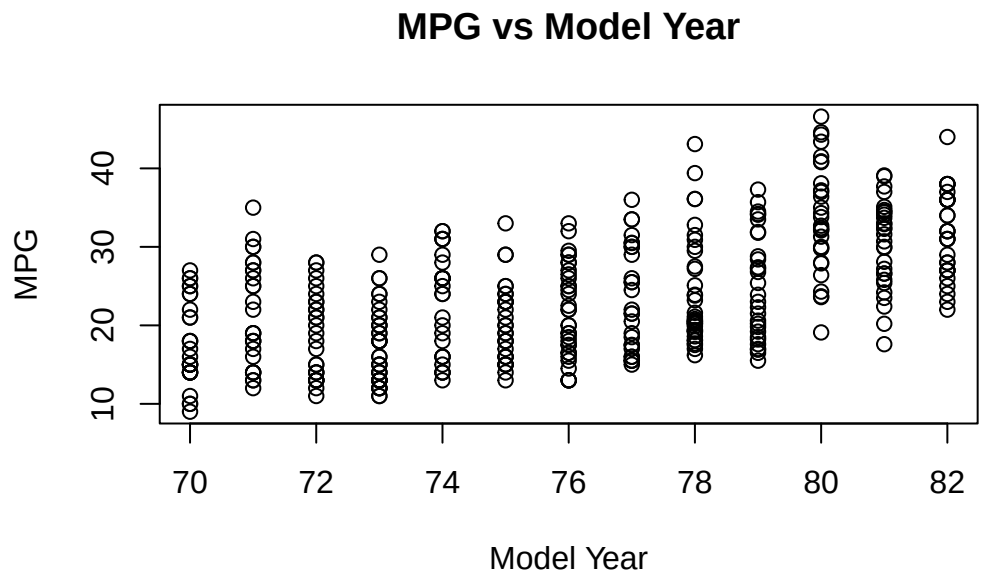

```
plot(auto$horsepower, auto$mpg,
     xlab = "Horsepower", ylab = "MPG",
     main = "MPG vs Horsepower")
```



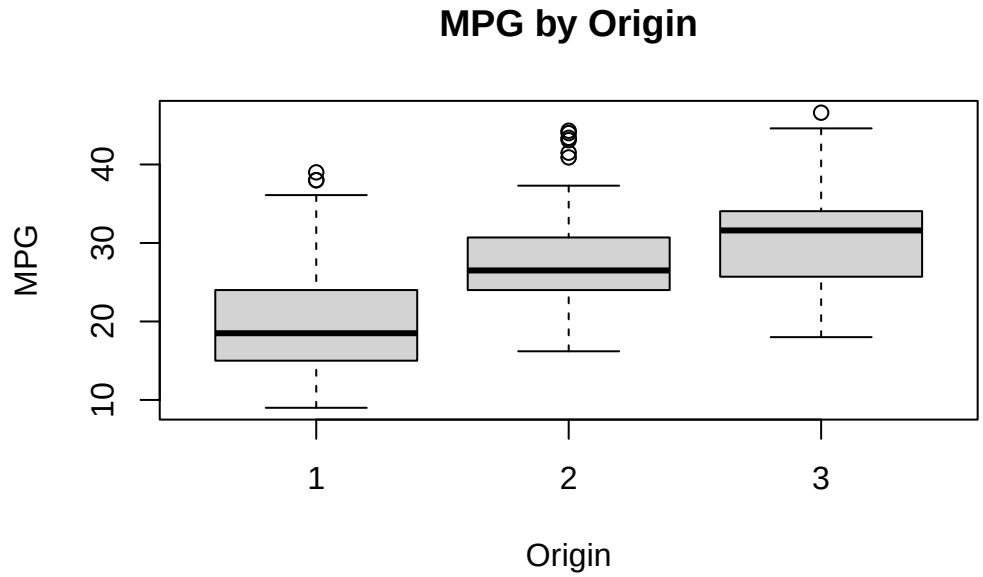

```
plot(auto$displacement, auto$mpg,
     xlab = "Displacement", ylab = "MPG",
     main = "MPG vs Displacement")
```




```
plot(auto$model_year, auto$mpg,
     xlab = "Model Year", ylab = "MPG",
     main = "MPG vs Model Year")
```




```r
boxplot(mpg ~ factor(origin), data = auto,
     xlab = "Origin", ylab = "MPG",
     main = "MPG by Origin")
```




Horsepower also shows a negative relationship, with slight curvature.

Displacement shows a negative relationship.

Model year shows a positive relationship (newer cars → higher mpg).

Origin shows a positive relationship with mpg (cars from higher-coded origins → higher mpg), with differences across groups indicating variation in fuel efficiency by origin.

**(ii) Report a correlation table of all variables, rounding to two decimal places (in the cor() function, set use="pairwise.complete.obs" to handle missing values)**

```
cor_table <- round(cor(auto_num, use = "pairwise.complete.obs"), 2)
cor_table
```

```
#>                mpg cylinders displacement horsepower weight acceleration
#> mpg           1.00     -0.78        -0.80      -0.78  -0.83         0.42
#> cylinders    -0.78      1.00         0.95       0.84   0.90        -0.51
#> displacement -0.80      0.95         1.00       0.90   0.93        -0.54
#> horsepower   -0.78      0.84         0.90       1.00   0.86        -0.69
#> weight       -0.83      0.90         0.93       0.86   1.00        -0.42
#> acceleration  0.42     -0.51        -0.54      -0.69  -0.42         1.00
#> model_year    0.58     -0.35        -0.37      -0.42  -0.31         0.29
#> origin        0.56     -0.56        -0.61      -0.46  -0.58         0.21
#>              model_year origin
#> mpg                0.58   0.56
#> cylinders         -0.35  -0.56
#> displacement      -0.37  -0.61
#> horsepower        -0.42  -0.46
#> weight            -0.31  -0.58
#> acceleration       0.29   0.21
#> model_year         1.00   0.18
#> origin             0.18   1.00
```

MPG is strongly negatively correlated with weight (-0.83), displacement (-0.80), and horsepower (-0.78), and positively correlated with model year (0.58) and origin (0.56).

**(iii) From the visualizations and correlations, which variables appear to relate to mpg?**

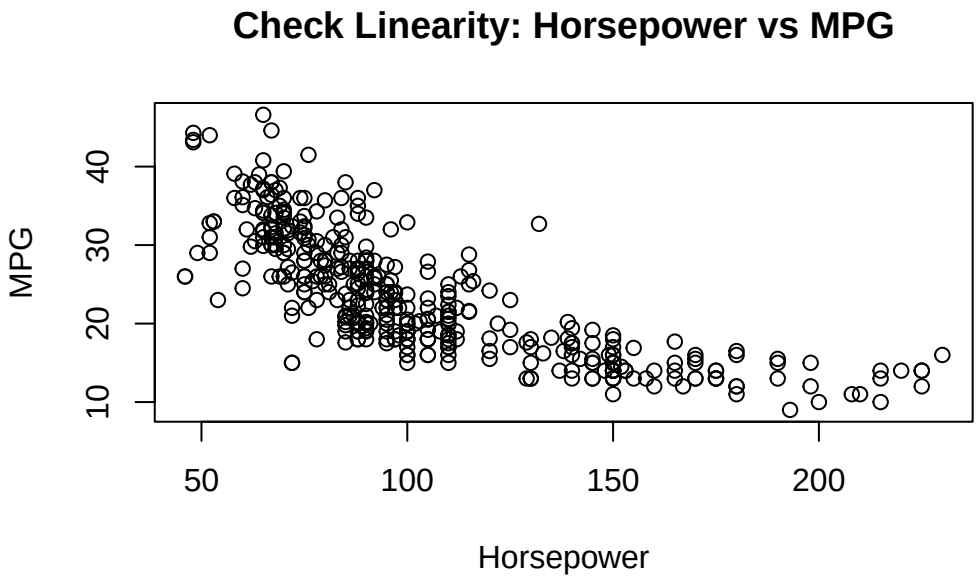
```
mpg_cor <- sort(cor_table["mpg", ], decreasing = TRUE)
mpg_cor
```

```
#>          mpg   model_year       origin acceleration    cylinders   horsepower
#>         1.00         0.58         0.56         0.42        -0.78        -0.78
#> displacement       weight
#>        -0.80        -0.83
```

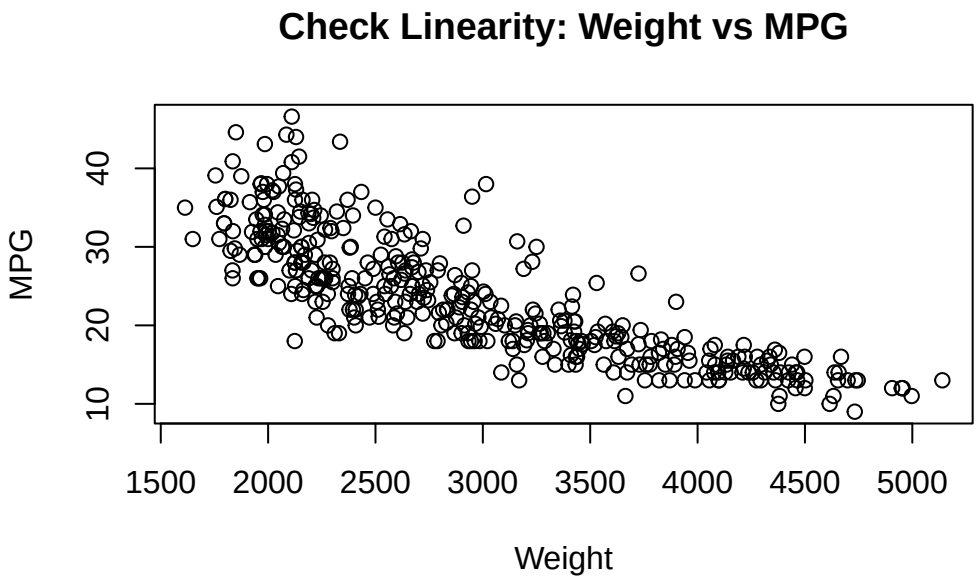
- Weight has the strongest negative relationship with mpg, followed by displacement and horsepower.
- Model year and origin show moderate positive relationships.
- Acceleration shows a weaker positive relationship.

**(iv) Which relationships might not be linear? (don't worry about linearity for rest of this HW)**

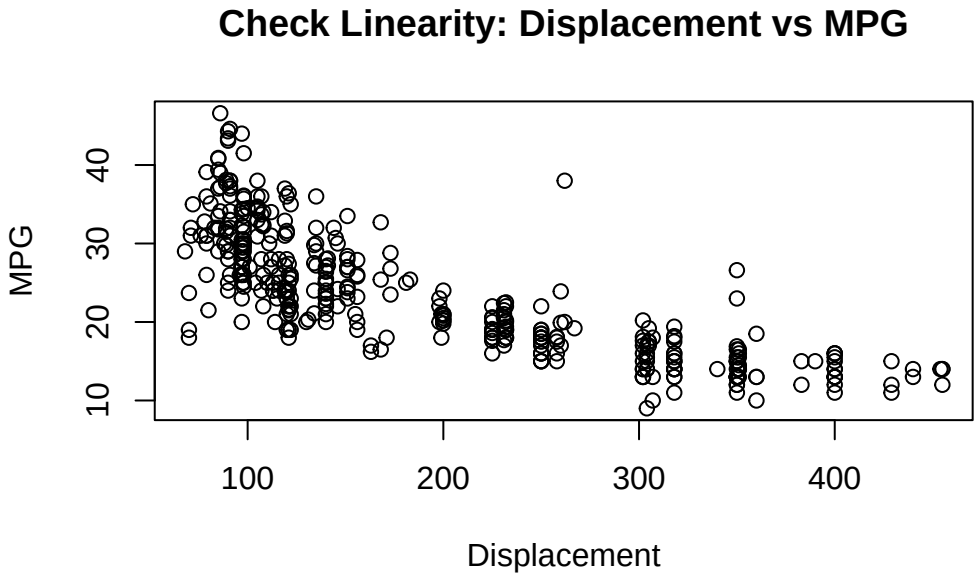
```
plot(auto$horsepower, auto$mpg,
     xlab = "Horsepower", ylab = "MPG",
     main = "Check Linearity: Horsepower vs MPG")
```




```
plot(auto$weight, auto$mpg,
     xlab = "Weight", ylab = "MPG",
     main = "Check Linearity: Weight vs MPG")
```




```r
plot(auto$displacement, auto$mpg,
     xlab = "Displacement", ylab = "MPG",
     main = "Check Linearity: Displacement vs MPG")
```




Horsepower vs mpg shows a non-linear (curved) pattern, indicating diminishing effects.

Weight and displacement appear more linear but still show slight curvature.

**(v) Are there any pairs of independent variables that are highly correlated (r > 0.7)?**

```
# remove mpg because we only check independent variables
ind_cor <- cor(auto_num[, -1], use = "pairwise.complete.obs")

# find pairs with |r| > 0.7
high_cor <- which(abs(ind_cor) > 0.7 & abs(ind_cor) < 1, arr.ind = TRUE)

# show unique pairs only
high_cor <- high_cor[high_cor[, 1] < high_cor[, 2], ]

data.frame(
  var1 = rownames(ind_cor)[high_cor[, 1]],
  var2 = colnames(ind_cor)[high_cor[, 2]],
  r = round(ind_cor[high_cor], 2)
)
```

```
#>           var1         var2    r
#> 1    cylinders displacement 0.95
#> 2    cylinders   horsepower 0.84
#> 3 displacement   horsepower 0.90
#> 4    cylinders       weight 0.90
#> 5 displacement       weight 0.93
#> 6   horsepower       weight 0.86
```

Several independent variables are highly correlated (r > 0.7), like the pairs that are output above, indicating multicollinearity.

This means these variables may contain redundant information, which can affect regression coefficient stability.

**(b) create a linear regression model where mpg is dependent upon all other suitable variables (Note: origin is categorical with three levels, so use factor(origin) in lm(...) to split it into two dummy variables)**

```
regr_auto <- lm(mpg ~ cylinders + displacement + horsepower + weight +
                  acceleration + model_year + factor(origin),
                data = auto)

summary(regr_auto)
```

```
#>
#> Call:
#> lm(formula = mpg ~ cylinders + displacement + horsepower + weight +
#>     acceleration + model_year + factor(origin), data = auto)
#>
#> Residuals:
#>     Min      1Q  Median      3Q     Max
#> -9.0095 -2.0785 -0.0982  1.9856 13.3608
#>
#> Coefficients:
#>                   Estimate Std. Error t value Pr(>|t|)
#> (Intercept)     -1.795e+01  4.677e+00  -3.839 0.000145 ***
#> cylinders       -4.897e-01  3.212e-01  -1.524 0.128215
#> displacement     2.398e-02  7.653e-03   3.133 0.001863 **
#> horsepower      -1.818e-02  1.371e-02  -1.326 0.185488
#> weight          -6.710e-03  6.551e-04 -10.243  < 2e-16 ***
#> acceleration     7.910e-02  9.822e-02   0.805 0.421101
#> model_year       7.770e-01  5.178e-02  15.005  < 2e-16 ***
#> factor(origin)2  2.630e+00  5.664e-01   4.643 4.72e-06 ***
#> factor(origin)3  2.853e+00  5.527e-01   5.162 3.93e-07 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 3.307 on 383 degrees of freedom
#>   (6 observations deleted due to missingness)
#> Multiple R-squared:  0.8242, Adjusted R-squared:  0.8205
#> F-statistic: 224.5 on 8 and 383 DF,  p-value: < 2.2e-16
```

**(i) Which independent variables have a 'significant' relationship with mpg at 1% significance?**

The independent variables displacement, weight, model year, and origin (the ones with p-value $<$ 0.01) have a statistically significant relationship with mpg at the 1 percent significance level.

```
coef_table <- summary(regr_auto)$coefficients

# variables with p-value < 0.01
coef_table[-1, ][coef_table[-1, "Pr(>|t|)"] < 0.01, ] # filter out intercept
```

```
#>                    Estimate   Std. Error    t value     Pr(>|t|)
#> displacement     0.023978644 0.0076532690   3.133124 1.862685e-03
#> weight          -0.006710384 0.0006551331 -10.242779 6.375633e-22
#> model_year       0.777026939 0.0517840867  15.005130 2.332943e-40
#> factor(origin)2  2.630002360 0.5664146647   4.643246 4.720373e-06
#> factor(origin)3  2.853228228 0.5527363020   5.162006 3.933208e-07
```

**(ii) Looking at the coefficients, is it possible to determine which independent variables are the most effective at increasing mpg? If so, which ones, and if not, why not? (hint: units!)**

```
coef(regr_auto)
```

```
#>      (Intercept)        cylinders     displacement       horsepower           weight
#>    -17.954602067     -0.489709424      0.023978644     -0.018183464     -0.006710384
#>     acceleration       model_year factor(origin)2 factor(origin)3
#>      0.079103036      0.777026939      2.630002360      2.853228228
```

> It is not appropriate to compare raw coefficients directly because the variables are measured in different units.

Therefore, we cannot determine which variable is most effective based on this model alone.

**(c) Try to resolve some of the issues with our regression model above:**

**(i) Create fully standardized regression results: are these slopes easier to compare?**

> Yes, the slopes are now easier to compare.

This is because all variables have been standardized to the same scale (mean = 0, standard deviation = 1), so the coefficients represent the relative importance of each variable. However, categorical variables (origin) should be interpreted separately.

```
auto_std <- auto

# standardize numeric variables only
vars_to_scale <- c("mpg", "cylinders", "displacement", "horsepower",
                   "weight", "acceleration", "model_year")

auto_std[, vars_to_scale] <- scale(auto_std[, vars_to_scale])

# do not standardize origin because it is categorical
regr_std <- lm(mpg ~ cylinders + displacement + horsepower + weight +
                 acceleration + model_year + factor(origin),
               data = auto_std)

summary(regr_std)
```

```
#>
#> Call:
#> lm(formula = mpg ~ cylinders + displacement + horsepower + weight +
```

```
#>     acceleration + model_year + factor(origin), data = auto_std)
#>
#> Residuals:
#>      Min       1Q   Median       3Q      Max
#> -1.15270 -0.26593 -0.01257  0.25404  1.70942
#>
#> Coefficients:
#>                 Estimate Std. Error t value Pr(>|t|)
#> (Intercept)     -0.13323    0.03174  -4.198 3.35e-05 ***
#> cylinders       -0.10658    0.06991  -1.524  0.12821
#> displacement     0.31989    0.10210   3.133  0.00186 **
#> horsepower      -0.08955    0.06751  -1.326  0.18549
#> weight          -0.72705    0.07098 -10.243  < 2e-16 ***
#> acceleration     0.02791    0.03465   0.805  0.42110
#> model_year       0.36760    0.02450  15.005  < 2e-16 ***
#> factor(origin)2  0.33649    0.07247   4.643 4.72e-06 ***
#> factor(origin)3  0.36505    0.07072   5.162 3.93e-07 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 0.423 on 383 degrees of freedom
#>   (6 observations deleted due to missingness)
#> Multiple R-squared:  0.8242, Adjusted R-squared:  0.8205
#> F-statistic: 224.5 on 8 and 383 DF,  p-value: < 2.2e-16
```

Weight has the largest magnitude coefficient, indicating it has the strongest effect on mpg. Model_year also has a strong positive effect. The origin dummy variables (origin 2 and origin 3) show that cars from these groups have higher mpg compared to the baseline group (origin 1).

**(ii) Regress mpg over each nonsignificant independent variable, individually. Which ones become significant when we regress mpg over them individually?**

> Cylinders, horsepower, and acceleration become significant when used individually because due to multicollinearity, their effects are masked in the full model, but when considered individually, their association with mpg becomes significant.

When isolated, their individual relationships with mpg become clearer. Proof:

```r
# check only main numeric variables
main_vars <- c("cylinders", "displacement", "horsepower", "weight",
               "acceleration", "model_year")

pvals <- coef_table[main_vars, "Pr(>|t|)"]

# variables not significant at 1% in the full model
```

```r
nonsig_vars <- names(pvals[pvals >= 0.01])
nonsig_vars
```

```
#> [1] "cylinders"    "horsepower"   "acceleration"
```

```r
# test each nonsignificant variable alone
for (v in nonsig_vars) {
  model <- lm(as.formula(paste("mpg ~", v)), data = auto)

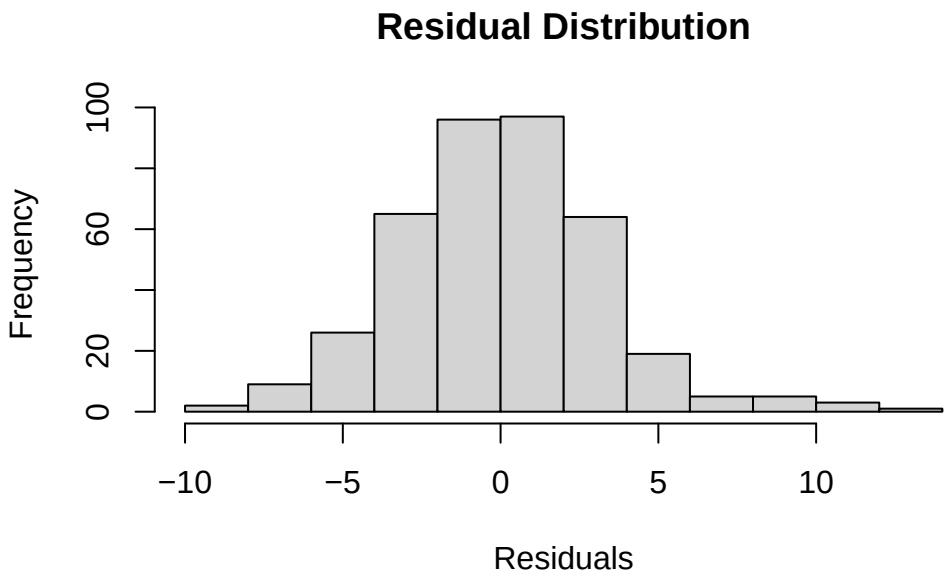
  print(paste("Single-variable model:", v))
  print(summary(model)$coefficients)
}
```

```
#> [1] "Single-variable model: cylinders"
#>              Estimate Std. Error   t value      Pr(>|t|)
#> (Intercept) 42.949314  0.8330414  51.55724 9.293816e-178
#> cylinders   -3.562887  0.1458098 -24.43516  4.503992e-81
#> [1] "Single-variable model: horsepower"
#>               Estimate  Std. Error   t value      Pr(>|t|)
#> (Intercept) 39.9358610 0.717498656  55.65984 1.220362e-187
#> horsepower  -0.1578447 0.006445501 -24.48914  7.031989e-81
#> [1] "Single-variable model: acceleration"
#>              Estimate Std. Error  t value     Pr(>|t|)
#> (Intercept)  4.969793  2.0432079 2.432348 1.544349e-02
#> acceleration 1.191205  0.1292364 9.217250 1.823092e-18
```

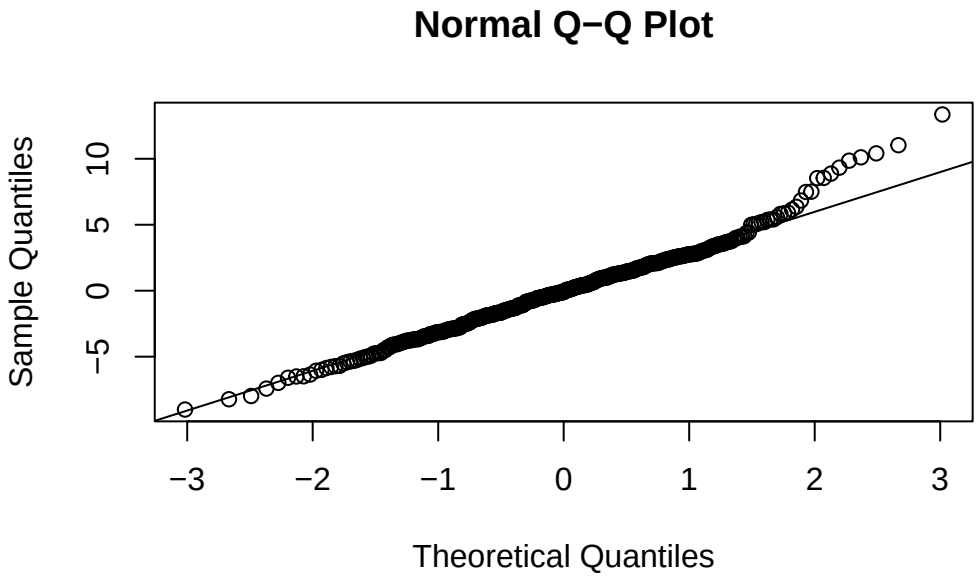
**(iii) Plot the distribution of the residuals: are they normally distributed and centered around zero?(get the residuals of a fitted linear model, e.g. regr <- lm(...), using regr$residuals)**

```r
res_auto <- regr_auto$residuals

# histogram checks whether residuals are centered around 0
hist(res_auto,
     main = "Residual Distribution",
     xlab = "Residuals")
```




```
# Q-Q plot checks whether residuals look normal
qqnorm(res_auto)
qqline(res_auto)
```




```
# mean should be very close to 0
mean(res_auto, na.rm = TRUE)
```

```
#> [1] 2.286837e-17
```

The residuals are centered around zero and appear approximately normally distributed, although slight deviations from normality are observed in the tails.